Victor Li under the supervision of Dr. Sriram Chandrasekaran
BIOMEDE490, Fall 2025
Dec 12, 2025

INDIGO–MTB Project Report (EHW4 and Related Analyses)

Overview:

Introduction:

The project goal was to use INDIGO-MTB to analyze drug combinations with EHW4 (and later with others). The process began with appending and normalizing a new condition (EHW4) to the MTB transcriptomic matrix. Then, the model would be trained/retrained with updated training data, and interaction predictions would be generated (2, 3, and a prioritised 4-way prediction) with correlation analyses. We later also integrated two other data sets (PZA and POA), which were provided as fitness tables, and repeated the correlation and predictions.

Background:

INDIGO (INferring Drug Interactions using chemoGenomics and Orthology) is an algorithm that predicts whether a drug combination will be synergistic or antagonistic without the need for experimental testing of every combination. Since testing drug combinations requires a significant amount of time and resources, especially when introducing more combinations (for example, a 4-drug combination with 100 drugs would necessitate millions of experiments), a model is needed to prioritise the most promising candidates for additional testing. INDIGO addresses this by using omics-based drug response profiles (originally chemogenomic profiles) together with a

set of experimentally measured interaction outcomes to learn patterns that correlate with synergy and antagonism.

Conceptually, INDIGO treats each drug as a drug-gene interaction profile and then builds a joint profile for a drug combo. INDIGO converts each pair of drugs into features that represent what the drugs do in common (sigma) and what they do differently (delta), and the model learns to map those features to an interaction score. The result is a trained INDIGO model that can take drug response profiles for drugs and output a predicted interaction score.

INDIGO-MTB is the tuberculosis-specific version of INDIGO that uses the M. tuberculosis drug response transcriptomes as the main input. The model is trained on a set of experimentally measured MTB drug interaction outcomes, which are reported as FIC or DiaMOND style scores. INDIGO-MTB produces interaction scores that can be interpreted as < 0.9, indicating synergy, 0.9-1.1 indicating additivity, and > 1.1 indicating antagonism.

Because the MTB transcriptomic matrix is assembled from multiple experiments and sources, INDIGO-MTB relies on batch correction (ComBat) to normalize expression profiles across conditions before doing any training/predicting. Additionally, INDIGO-MTB is not limited to pairwise predictions, as you will see later.

MATLAB Data and Code

Data:

- averaged_data_new.mat
- EHW4 vs Mtb H37Rv_SW.xlsx
- Tb_training.xlsx
- Tb_training_plus.xlsx
- Tb_training_plus_updated.xlsx
- Identifiers_match_tb.xlsx
- sam_FIC.xlsx
- sam_FIC_Updated.xlsx
- PZA Fitness.csv
- POA Fitness.csv

Intermediate Datasets:

- normData_EHW4.mat: ComBat-corrected matrix after appending EHW4
- normData_EHW4_PZA_POA.mat: ComBat-corrected matrix after adding PZA and POA.
- identifiers_match_tb_completed.xlsx: filled map ensuring every column in the ComBat matrix has a drug ID.

Code:

- ComBat.m

- Process_transcriptome_tb.m
- Indigo_train_tb.m
- Indigo_predict_tb.m
- fitnessFile_to_column.m

Predict Initial EHW4 Combos (with ComBat)

Objectives: Append EHW4 LogFC to the master matrix, fix the single sample batch ComBat, build the phenotype (z=2), and predict EHW4 vs all drugs.

First, load the master matrix.

```
S = load(fullfile(dataDir,'averaged_data_new.mat'),
        'averagedtbdata','mtb_expression_database_row_ids','mtb_expression_database_col_ids');
X    = double(S.averagedtbdata);
rows = string(S.mtb_expression_database_row_ids(:));
cols = string(S.mtb_expression_database_col_ids(:));
```

Next, build and append the EHW4 column. We align using the RV IDs and we put 0's for missing values.

```
T = readtable(fullfile(dataDir,'EHW4 vs Mtb H37Rv_SW (1).xlsx'),'TextType','string');
vars  = string(T.Properties.VariableNames);
gix   = find(contains(lower(vars),'gene'),1); if isempty(gix), gix = 1; end
genes = string(T{:,gix});
if any(strcmpi(vars,'logFC')), logFC = double(T.('logFC'));
else
    isNum = varfun(@isnumeric,T,'OutputFormat','uniform');
    logFC = double(T.(vars(find(isNum,1))));
end
[U,~,ic] = unique(upper(genes));
valsU    = accumarray(ic, logFC, [], @mean);
newCol   = zeros(numel(rows),1);
[tf,loc] = ismember(upper(U), upper(rows));
newCol(loc(tf)) = valsU(tf);
newCol(~isfinite(newCol)) = 0;
X2    = [X, newCol];
cols2 = [cols; "EHW4"];
```

We clean the matrix by replacing NaN values with the median of the column so it doesn't affect our predictions.

```
X2c = X2;
for j = 1:size(X2c,2)
    c = X2c(:,j); m = median(c(~isnan(c))); if isnan(m), m = 0; end
    c(isnan(c)) = m; X2c(:,j) = c;
end
```

Now, we will use ComBat to normalize the data. However, ComBat requires 2 columns per batch, and we only have 1 column in the EHW4 batch, so we worked around that by duplicating EHW4 with a very small amount of noise.

```
idxE = find(strcmpi(cols2,'EHW4'),1);
E    = X2c(:,idxE);
epsJ = 1e-6 * max(1, std(E));                % tiny jitter
Xtmp = [X2c, E + epsJ*randn(size(E))];
colsT = [cols2; "EHW4_dup"];
batch = ones(size(Xtmp,2),1); batch([idxE end]) = 2;

normData_tmp = ComBat(Xtmp, batch, [], 1);
normData = normData_tmp(:,1:end-1);          % drop the duplicated column
cols_cb  = colsT(1:end-1);
```

Now, we build the phenotype (z=2) and check that the identifiers map cover every column.

```
z = 2;
[phenotype_data, phenotype_labels] = process_transcriptome_tb(normData, cellstr(rows), z);

id0 = readtable(fullfile(dataDir,'identifiers_match_tb.xlsx'),'TextType','string');
id0.Properties.VariableNames(1:2) = {'DrugID','ColLabel'};
missing = setdiff(cols_cb, string(id0.ColLabel), 'stable');
if ~isempty(missing)
    addRows = table(missing, missing, 'VariableNames', {'DrugID','ColLabel'});
    idMap = [id0; addRows];
    key = strcat(string(idMap.DrugID),'→',string(idMap.ColLabel));
    [~,k] = unique(key,'stable'); idMap = idMap(k,:);
else
    idMap = id0;
end
idmapFile = fullfile(dataDir,'identifiers_match_tb_completed.xlsx');
writetable(idMap, idmapFile);
```

Now we can train on the original MTB training and predict EHW4 vs all drugs.

```
[train_scores, train_pairs] = xlsread(fullfile(dataDir,'tb_training.xlsx'));
[~, ~, ~, indigo_model, ~, ~] = indigo_train_tb(
    [], idmapFile, [], 1,
    phenotype_data, phenotype_labels, cellstr(cols_cb),
    train_scores, train_pairs);

Tmap = readtable(idmapFile,'TextType','string'); Tmap.Properties.VariableNames(1:2) = {'DrugID','ColLabel'}
keep  = ismember(Tmap.ColLabel, cols_cb);
drugIDsPresent = unique(Tmap.DrugID(keep),'stable');
partners = setdiff(drugIDsPresent, "EHW4", 'stable');
pairs    = [repmat({'EHW4'}, numel(partners),1), cellstr(partners(:))];

[pI, pS] = indigo_predict_tb(
    indigo_model, pairs, 2, idmapFile, [], 1,
    phenotype_data, phenotype_labels, cellstr(cols_cb));

Tout = cell2table(pI,'VariableNames',{'DrugA','DrugB'}); Tout.Score = pS;
outFile = fullfile(outDir,'EHW4_Predictions_ComBat.xlsx');
writetable(Tout, outFile);
```

Results:

We found the top 10 most synergistic scores pictured below (<1 means synergistic):

```
{'EHW4'}    {'CPZ'          }    {[0.68105]}
{'EHW4'}    {'GSNO/CPZ'     }    {[0.72159]}
{'EHW4'}    {'ASCIDIDEMIN'  }    {[0.74118]}
{'EHW4'}    {'VERAPAMIL'    }    {[0.75691]}
{'EHW4'}    {'TRZ'          }    {[0.76519]}
{'EHW4'}    {'IMTB001'      }    {[0.76658]}
{'EHW4'}    {'ECONAZOLE'    }    {[0.77173]}
{'EHW4'}    {'GSNO/CFZ'     }    {[0.79122]}
{'EHW4'}    {'CLARYTHROMYCIN'}   {[0.80203]}
{'EHW4'}    {'TRICLOSAN'    }    {[0.80364]}
```

We also found the top 10 most antagonistic scores pictures below (>1 means antagonistic):

```
{'EHW4'}    {'Moxifloxacin'}    {[1.9797]}
{'EHW4'}    {'SM'          }    {[1.3923]}
{'EHW4'}    {'ETA'         }    {[1.3623]}
{'EHW4'}    {'IMTB012'     }    {[1.3073]}
{'EHW4'}    {'IMTB014'     }    {[1.2759]}
{'EHW4'}    {'VANCOMYCIN'  }    {[ 1.259]}
{'EHW4'}    {'IMTB047'     }    {[1.2398]}
{'EHW4'}    {'IMTB045'     }    {[1.2383]}
{'EHW4'}    {'OFLOX'       }    {[1.2266]}
{'EHW4'}    {'Capreomycin' }    {[1.2257]}
```

We also saved the normalized matrix that we generated earlier for later use.

Retrain the Model with Additional Data

First, we appended the additional data, and afterwards, we would need to retrain the INDIGO model with the appended training data.

```
trainX = fullfile(dataDir,'tb_training_plus_updated.xlsx');
[train_scores, train_pairs] = xlsread(trainX);

[~, ~, ~, modelNew, ~, ~] = indigo_train_tb(
    [], idmapFile, [], 1,
    phenotype_data, phenotype_labels, cellstr(cols_cb),
    train_scores, train_pairs);
```

Afterwards, through the same methodology as before, we predicted EHW4 vs all drugs with the retrained INDIGO model.

Results:

After retraining, we expected some differences between the old scores and the new ones; however, these scores looked relatively similar after generating an Excel table of comparisons between the scores before and after.

2-Way and 3-Way Predictions with EHW4

The two-way predictions were already detailed above so only the methodology for the 3-way predictions with EHW4 will be depicted below.

The start of the 3-way prediction was relatively similar to the previous one.

1. Load the ComBat normalized matrix and labels as well.
2. Build the phenotype at z=2 and check that the identifiers map cover every column.
3. Train a fresh model of INDIGO.

After step 3 is where the method deviates. The 3-way combos take up a lot of RAM, and I ran into some issues on my computer, so instead of scoring all 3-way combos in one pass, I split the entire process into smaller chunks and appended the results.

```
others  = cellstr(setdiff(unique(readtable(idmapFile).DrugID), "EHW4", 'stable'));
others  = others(ismember(others, cellstr(unique(readtable(idmapFile).DrugID(ismember(readtable(idmapFile).ColLabel, cols_cb),'stable')))));
C       = nchoosek(others, 2);
batchSz = 3000;

T3 = table('Size',[0 4], 'VariableTypes',{'string','string','string','double'},
           'VariableNames',{'DrugA','DrugB','DrugC','Score'});

n = size(C,1);
for k = 1:batchSz:n
    r    = k:min(k+batchSz-1, n);
    trip = [repmat({'EHW4'}, numel(r),1), C(r,:)];
    [PI3, PS3] = indigo_predict_tb(
        modelNew, trip, 3,
        idmapFile, [], 1,
        phenotype_data, phenotype_labels, cellstr(cols_cb));
    chunk = cell2table(PI3,'VariableNames',{'DrugA','DrugB','DrugC'});
    chunk = convertvars(chunk, {'DrugA','DrugB','DrugC'}, 'string');
    chunk.Score = double(PS3(:));
    T3 = [T3; chunk];
    if mod(r(end), 5000)==0 || r(end)==n
        fprintf('3-way', r(end), n);
    end
end

file3 = fullfile(outDir,'EHW4_3way_retrained_updated.xlsx');
writetable(T3, file3);
```

Results:

The top 10 most synergistic and antagonistic combos are depicted below (in order):

DrugA	DrugB	DrugC	Score
EHW4	AMIKACIN	VERx	0.456392616
EHW4	CLOFAZIMINE	INH	0.496201597
EHW4	INH	VERx	0.504149786
EHW4	TET	VERx	0.510234087
EHW4	AMIKACIN	THZ (6hrMIC)	0.511229285
EHW4	CPZ	INH	0.511801615
EHW4	PA824	RIF	0.516805683
EHW4	SULFAMETHIZOLE	VERx	0.518613017
EHW4	IMTB031	VERx	0.51925639
EHW4	ACTINOMYCIND	VERx	0.529333984

DrugA	DrugB	DrugC	Score
EHW4	HN878_rif_resistant	TKK_010033_ethionamide_resistant	2.828425677
EHW4	2169Uganda_rifampicin_resistant	TKK_010040_ethionamide_resistant	2.706873558
EHW4	TRS_OFX_resistant	TKK_010040_ethionamide_resistant	2.696094089
EHW4	ETA	TKK_010040_ethionamide_resistant	2.673925944
EHW4	HN878_rif_resistant	TKK_010025_ethionamide_resistant	2.658161539
EHW4	EMB	TKK_010040_ethionamide_resistant	2.652894581
EHW4	OFX1	TKK_010040_ethionamide_resistant	2.647963511
EHW4	CERULENIN	TKK_010040_ethionamide_resistant	2.646933739
EHW4	SM	TKK_010040_ethionamide_resistant	2.644741088
EHW4	Moxifloxacin	TKK_010040_ethionamide_resistant	2.639849389

Correlation Analyses on EHW4

Our objective here is to rank all conditions by similarity to EHW4 using both Pearson and Spearman correlations. Afterwards, we will compute the p-values for each correlation.

```
>> S = load(fullfile(dataDir,'normData_EHW4.mat'));
Xcb  = double(S.normData); rows = string(S.rows); cols = string(S.cols_cb);

iE = find(strcmpi(cols,'EHW4'),1); assert(~isempty(iE),'some empty');
e  = Xcb(:,iE);

rhoS = zeros(numel(cols),1); pS = zeros(numel(cols),1);
rhoP = zeros(numel(cols),1); pP = zeros(numel(cols),1);

for j = 1:numel(cols)
    [rhoS(j), pS(j)] = corr(e, Xcb(:,j), 'Type','Spearman','Rows','pairwise');
    [rhoP(j), pP(j)] = corr(e, Xcb(:,j), 'Type','Pearson' ,'Rows','pairwise');
end

Tcorr = table(cols, rhoS, pS, rhoP, pP,
    'VariableNames', {'Condition','Spearman','Spearman_p','Pearson','Pearson_p'});

TcorrS = sortrows(Tcorr, 'Spearman','descend');
writetable(TcorrS, fullfile(outDir,'EHW4_similarity_by_rankcorr_withP.xlsx'));

TcorrP = sortrows(Tcorr, 'Pearson','descend');
writetable(TcorrP, fullfile(outDir,'EHW4_correlation_pearsonWithP.xlsx'));

disp(TcorrS(1:10,:));
disp(TcorrP(1:10,:));
```

Results:

We got very low Pearson correlations and also very small p-values, which is to be expected because we are correlating very long vectors.

4-Way Predictions with EHW4 (Top-35 list)

Our objective here is to predict 4-way drug combinations (EHW4, B, C, D) with EHW4. We decided to use a list of the Top-35 highest priority drugs from the INDIGO MTB paper.

To accomplish this:

1. We use the retrained INDIGO model and the same phenotype and mapping as before.
2. We number all triplets from the top 35 list and also anchor EHW4.

3. Next we will use the indigo_predict_tb function with inputtype = 4 (basically like 4 drug combo mode) and also batch the calls like we did before with the 3-way predictions.

Implementation:

```
top35 = string({
"Amikacin","Amoxicillin","Azithromycin","Bedaquiline","Capreomycin","Cefoxitin",
"Chlorpromazine","Ciprofloxacin","Clarythromycin","Clofazimine","Dapsone","Delamanid",
"Ethambutol","Ethionamide","Isoniazid","Kanamycin","Linezolid","Methotrexate",
"Minocycline","Moxifloxacin","Norfloxacin","Ofloxacin","P218","PBTZ169",
"Pretomanid","Rifampicin","Rifapentine","Spectinomycin","Streptomycin",
"Sulfamethoxazole","Sutezolid","Tetracycline","Thioridazine","Trimethoprim","Verapamil"});

Tmap = readtable(idmapFile,'TextType','string'); Tmap.Properties.VariableNames(1:2) = {'DrugID','ColLabel'};
keep = ismember(Tmap.ColLabel, cols_cb); drugIDsPresent = unique(Tmap.DrugID(keep),'stable');
top35 = top35(ismember(upper(top35), upper(drugIDsPresent)));

C = nchoosek(cellstr(top35), 3);       % all (B,C,D) combs
batchSz = 3000;

T4 = table('Size',[0 5], 'VariableTypes',{'string','string','string','string','double'},
          'VariableNames',{'DrugA','DrugB','DrugC','DrugD','Score'});
T4 = table('Size',[0 5], 'VariableTypes',{'string','string','string','string','double'},
           'VariableNames',{'DrugA','DrugB','DrugC','DrugD','Score'});

n = size(C,1);
for k = 1:batchSz:n
    r    = k:min(k+batchSz-1, n);
    quad = [repmat({'EHW4'}, numel(r),1), C(r,:)];
    [PI4, PS4] = indigo_predict_tb(
        modelNew, quad, 4,
        idmapFile, [], 1,
        phenotype_data, phenotype_labels, cellstr(cols_cb));
    chunk = cell2table(PI4,'VariableNames',{'DrugA','DrugB','DrugC','DrugD'});
    chunk = convertvars(chunk, {'DrugA','DrugB','DrugC','DrugD'}, 'string');
    chunk.Score = double(PS4(:));
    T4 = [T4; chunk];
    if mod(r(end), 2000)==0 || r(end)==n
        fprintf('done', r(end), n);
    end
end

outFile = fullfile(outDir,'EHW4_4way_TOP35.xlsx');
```

We are essentially defining a list of drugs, mapping & filtering them so that we only keep drugs present in the ComBat matrix and the identifier map, and then generating all 3-drug subsets from that filtered list. For each subset, we form a 4-drug combo with EHW4 as drug A. We then score each combo using indigo_predict_tb, and these scores are appended to the score column after DrugD in the results file.

Results:

The 10 most synergistic 4-way drug combinations that we found were:

DrugA	DrugB	DrugC	DrugD	Score
EHW4	BDQ	CLOFAZIMINE	INH	0.399834805
EHW4	CLARYTHROMYCIN	INH	VERx	0.441966209
EHW4	BDQ	CLOFAZIMINE	RIF	0.449543487
EHW4	AZITHROMYCIN	INH	VERx	0.454354921
EHW4	DAPx	INH	VERx	0.468568355
EHW4	INH	SMXx	VERx	0.468568355
EHW4	CLOFAZIMINE	INH	RIF	0.468699127
EHW4	INH	P218x	VERx	0.468903094
EHW4	INH	MINOCYCLINE	VERx	0.470508321
EHW4	AMIKACIN	CLARYTHROMYCIN	VERx	0.470643158

Predict PZA and POA Combos (with correlation analyses)

We were given 2 new datasets, PZA and POA, and our objective was to:

1. Compute correlations (with p-values) vs all conditions
2. And also run a 2-way prediction vs all drugs with both PZA and POA

The implementation for this was much the same as the process outlined with EHW4. We parsed the PZA and POA CSVs and mapped their measurements onto the MTB Rv gene list and appended them as new transcriptomic conditions in the existing matrix. This time, we can run ComBat directly without having to add an additional column with some noise because we have 2 samples, so we re-ran ComBat directly to batch-correct the combined dataset. Then, we rebuilt the phenotype at the same settings and retrained INDIGO using the updated matrix. After training, we computed correlation stats (with p-values) that compared PZA and POA with all conditions and finally ran 2-way predictions with PZA and POA with every other drug in the dataset.

Results:

The top 10 most synergistic combos for POA are depicted below:

DrugA	DrugB	Score
POA	VERx	0.505327374
POA	VER1	0.620974456
POA	CPZ	0.637698548
POA	THZ (1hrMIC)	0.662540701
POA	GSNO/CPZ	0.6923901
POA	IMTB032	0.709807902
POA	TRZ	0.717876993
POA	IMTB001	0.767845235
POA	VERAPAMIL	0.788484391
POA	THZ (6hr4MIC)	0.796801456

The top 10 most synergistic combos for PZA are depicted below:

DrugA	DrugB	Score
PZA	VERx	0.609452328
PZA	CPZ	0.613694105
PZA	GSNO/CPZ	0.664657583
PZA	IMTB001	0.68408768
PZA	VER1	0.700439014
PZA	THZ (1hrMIC)	0.713445971
PZA	TRZ	0.713658455
PZA	CLOFAZIMINE	0.749301142
PZA	THZ (6hrMIC)	0.753809331
PZA	THZ (6hr4MIC)	0.760829213

Our results for the correlation analyses suggest that PZA has clear similarities with other conditions in the database. The strongest matches are 5-CL-PZA, NAM, and BZA with high Spearman and Pearson correlations; dozens of conditions have Spearman> 0.3 for PZA. POA, on the other hand, only has modest correlations with the top Spearman being only 0.31 (MACROPHAGE), and most other values are much lower. We can interpret POA's similarity with the conditions in our data as much weaker than PZA. Notably, POA is strongly

anti-correlated with PZA, with a negative Spearman and a very negative Pearson. The p-values are again very small, likely due to the large number of genes.